

Diseño del sistema y arquitectura

Introducción al Diseño del Sistema

El **diseño del sistema** es una fase crítica en el desarrollo de software, ya que establece cómo se organizarán los diferentes componentes del sistema para cumplir con los requisitos funcionales y no funcionales.

Un buen diseño de sistema garantiza:

- **Escalabilidad:** El sistema puede crecer sin perder rendimiento.
- **Mantenibilidad:** Facilita la corrección de errores y la incorporación de nuevas funcionalidades.
- **Eficiencia:** Uso óptimo de los recursos computacionales.
- **Seguridad:** Protección de los datos y las interacciones del usuario.

El diseño del sistema se divide en varias áreas clave, cada una de las cuales cumple un rol fundamental en el funcionamiento del software:

1. **Base de datos** – Cómo se almacenan y gestionan los datos.
2. **Interfaz de usuario (UI)** – Cómo los usuarios interactúan con el sistema.
3. **Lógica de negocio** – Cómo se procesan los datos y se aplican las reglas de negocio.



Componentes Principales del Diseño del Sistema

Base de Datos

La base de datos es el núcleo del sistema, ya que almacena toda la información que el software necesita para funcionar correctamente.

Modelado de Datos

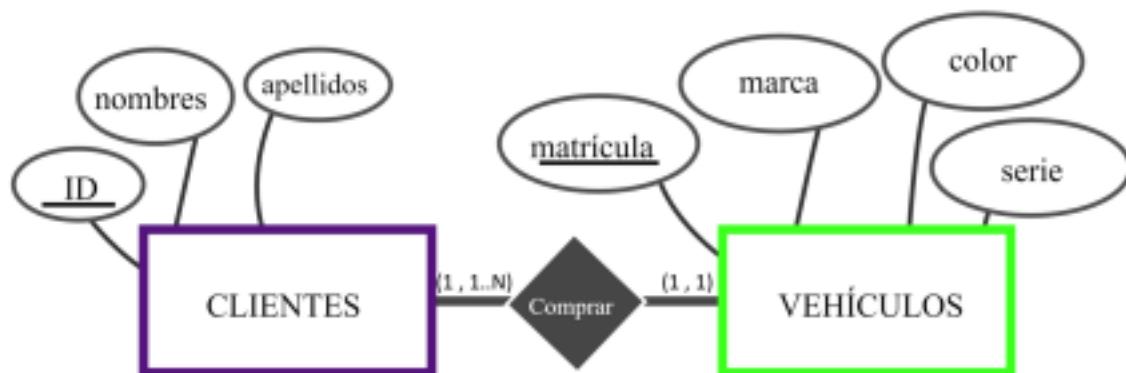
Antes de implementar una base de datos, es fundamental diseñar un **modelo de datos**, que

define qué información se almacenará y cómo se relacionarán los distintos elementos.

Uno de los enfoques más utilizados para modelar bases de datos relacionales es el **Modelo Entidad-Relación (E-R)**. En este modelo:

- **Entidades:** Representan objetos del mundo real (ej., "Usuario", "Producto", "Pedido").
- **Atributos:** Son las características de cada entidad (ej., "Nombre", "Edad", "Precio").
- **Relaciones:** Conectan diferentes entidades (ej., "Un usuario puede realizar varios pedidos").

Este modelo se representa gráficamente mediante **diagramas entidad-relación**, que ayudan a visualizar la estructura de la base de datos antes de su implementación.



Normalización

Es una técnica utilizada para organizar los datos en una base de datos relacional de manera eficiente, evitando redundancias e inconsistencias. Se realiza en varias **formas normales**, que establecen reglas para dividir grandes tablas en tablas más pequeñas y establecer relaciones entre ellas.

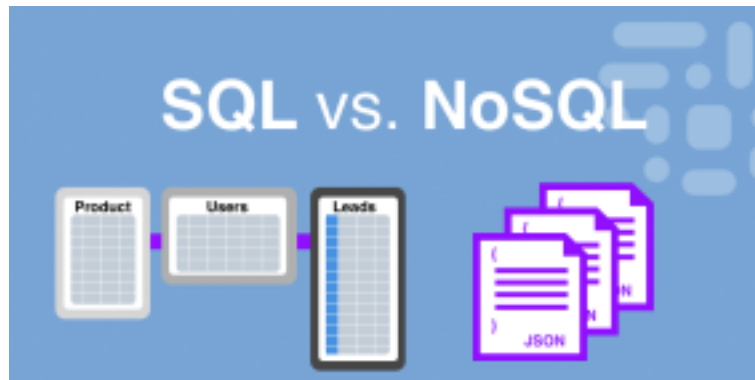
Las formas normales más utilizadas son:

- **Primera forma normal (1FN):** Eliminar grupos repetitivos y asegurar que cada campo tenga un único valor.
- **Segunda forma normal (2FN):** Asegurar que todos los atributos dependan completamente de la clave primaria.
- **Tercera forma normal (3FN):** Eliminar dependencias transitivas entre los atributos.

Tipos de Bases de Datos

Las bases de datos pueden clasificarse en dos grandes categorías:

- **Bases de datos relacionales (SQL):** Organizadas en tablas con claves primarias y foráneas, garantizan integridad y consistencia en los datos. Ejemplos: MySQL, PostgreSQL, SQL Server.
- **Bases de datos NoSQL:** Diseñadas para almacenar grandes volúmenes de datos no estructurados y facilitar la escalabilidad horizontal. Ejemplos: MongoDB (documentos), Redis (clave-valor), Neo4j (grafos).



Interfaz de Usuario (UI)

La **interfaz de usuario** es la parte visible del sistema con la que interactúan los usuarios. Un buen diseño de UI es esencial para garantizar una **experiencia de usuario (UX)** positiva.

Principios del Diseño de Interfaces

Un diseño de interfaz bien estructurado sigue principios básicos como:

- **Simplicidad:** La interfaz debe ser clara, sin elementos innecesarios.
- **Consistencia:** Debe mantener un estilo uniforme en colores, fuentes y botones.
- **Feedback inmediato:** Debe proporcionar respuestas visuales o auditivas a las acciones del usuario (ej., mensajes de éxito o error).
- **Accesibilidad:** Debe ser usable por personas con discapacidades, con opciones como contraste alto o compatibilidad con lectores de pantalla.

Wireframes y Prototipos

Antes de desarrollar la interfaz, es recomendable crear:

- **Wireframes:** Esquemas básicos que muestran la estructura de las pantallas sin detalles gráficos.
- **Prototipos:** Versiones interactivas que permiten probar la usabilidad antes de la implementación.

Existen herramientas como Figma, Adobe XD y Balsamiq que facilitan la creación de estos elementos.

Diseño responsivo

El **diseño responsivo** asegura que la interfaz de usuario se adapte a diferentes dispositivos y tamaños de pantalla. Esto es crucial en aplicaciones web y móviles.

- **Técnicas de diseño responsivo:** Uso de grids flexibles, imágenes escalables y media queries en CSS.



Lógica de Negocio

La **lógica de negocio** define las reglas y procesos que rigen el funcionamiento del sistema.

Ejemplo: En un sistema de e-commerce, la lógica de negocio define cómo calcular descuentos, validar medios de pago y gestionar envíos.

Patrones de Diseño en la Lógica de Negocio

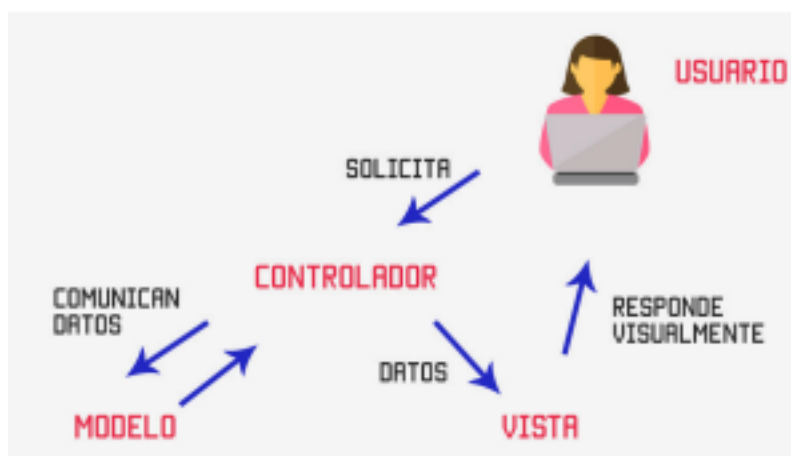
Para organizar el código de manera eficiente, se utilizan patrones de diseño que separan las responsabilidades del sistema.

MVC (Modelo-Vista-Controlador)

Uno de los patrones más utilizados en el desarrollo de aplicaciones es **MVC (Modelo-Vista-Controlador)**, que divide la aplicación en:

- **Modelo:** Gestiona los datos y la lógica de negocio.
- **Vista:** Representa la interfaz de usuario.
- **Controlador:** Conecta la vista con el modelo, procesando las acciones del usuario.

Alternativas a MVC incluyen **MVVM (Modelo-Vista-ViewModel)** y **MVP (Modelo-Vista-Presentador)**



Buenas prácticas en el diseño de la lógica del negocio

- **Separación de preocupaciones:** Dividir la lógica del negocio en módulos independientes.
- **Reutilización de código:** Evitar la duplicación de código mediante funciones y clases

reutilizables.

- **Pruebas unitarias:** Verificar que cada componente de la lógica del negocio funcione correctamente.

Arquitectura de Software

La **arquitectura del software** define la estructura general del sistema, incluyendo la organización de sus componentes y la forma en que interactúan entre sí.

Modelos Arquitectónicos Comunes

Existen varios modelos arquitectónicos, cada uno con ventajas y desventajas según el tipo de proyecto.

Arquitectura Monolítica

- Toda la aplicación se desarrolla como un único bloque de código.
- **Ventajas:** Sencilla de desarrollar y desplegar.
- **Desventajas:** Difícil de escalar y mantener a largo plazo.

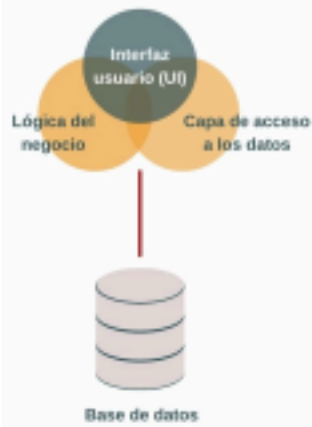
Arquitectura de Microservicios

- La aplicación se divide en servicios independientes que se comunican entre sí mediante APIs.
- **Ventajas:** Escalabilidad y facilidad para actualizar partes del sistema sin afectar al resto.
- **Desventajas:** Mayor complejidad en la gestión de servicios y comunicación entre ellos.

Patrón Cliente-Servidor

- Separación entre un **cliente** (interfaz de usuario) y un **servidor** (lógica de negocio y base de datos).
- Es el modelo más común en aplicaciones web y móviles.

Arquitectura monolítica



Arquitectura microservicios

